Group 9 Assignment Report

An Evolutionary Algorithm for the Travelling Salesman Problem

Christos Zampounis Mehmet Cihan Yalçın Péter Dávid Hajdu
Pierre Kasparian Prajval Parthiban

September 30, 2026

1 Introduction

The goal of this assignment is to design, implement and experimentally evaluate an evolutionary algorithm for a combinatorial optimisation problem. We chose the Travelling Salesman Problem, whose discrete, permutation-based search space and relevance to routing and logistics make it a natural and challenging testbed for evolutionary computation. This report presents our problem formulation, the evolutionary approach and operators we developed, and an experimental analysis of its solution quality and convergence behaviour.

2 The Travelling Salesman Problem

2.1 Mathematical Formulation

The Travelling Salesman Problem (TSP) is a classical NP-hard combinatorial optimisation problem: given a set of N cities and the distances between them, the task is to find the shortest closed tour that visits every city exactly once. Following the formulation of Potvin [1], we model the problem on a complete weighted graph $G = (V, E)$, where $V = \{1, \dots, N\}$ is the set of cities and d_{ij} is the distance between cities i and j. The decision variables are $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ if and only if the tour travels directly from city i to city j. The objective is to minimise the total travelled distance,

$$\text{Minimise} \quad \sum_{i=1}^{N} \sum_{j=1}^{N} d_{ij}\, x_{ij}, \tag{1}$$

subject to the degree constraints

$$\sum_{j=1}^{N} x_{ij} = 1, \quad i = 1, \dots, N, \qquad \sum_{i=1}^{N} x_{ij} = 1, \quad j = 1, \dots, N, \tag{2}$$

which require every city to have exactly one incoming and one outgoing edge, and the subtour-elimination constraints

$$\sum_{i,j \in S} x_{ij} \;\le\; |S| - 1, \qquad \forall\, S \subset V,\; 2 \le |S| \le N - 1, \tag{3}$$

which forbid partial cycles over any proper subset S of the cities.

Because an evolutionary algorithm operates on orderings rather than on binary edge variables, it is convenient to state the equivalent tour-as-permutation formulation. A candidate solution is a permutation $\pi = (\pi_1, \dots, \pi_N)$ of the cities, and its cost is the length of the cycle that visits them in that order,

$$C(\pi) \;=\; \sum_{k=1}^{N-1} d_{\pi_k, \pi_{k+1}} \;+\; d_{\pi_N, \pi_1}. \tag{4}$$

The evolutionary algorithm therefore seeks a permutation minimising Equation (4).

2.2 Related Evolutionary Approaches

Liu [2] builds a genetic algorithm around a crossover operator called *Edge Swapping* (ES), which keeps the edges shared by the two parent tours and swaps in the cycles where they disagree; seeding the population with greedy tours refined by 2-opt, it reaches best-known solutions on large benchmarks. Liao et al. [3] propose GPSO, a genetic-based particle swarm optimisation in which each particle is a real-valued vector decoded into a tour by ranking its components, initialised geometrically by sorting cities by their angle around the geographical centre, which produces a non-crossing route. For the path representation, where a chromosome is the visiting order and classical one-point or uniform crossovers cannot be used as they duplicate cities, Hussain et al. [4] compare partially mapped (PMX), order (OX) and cycle crossover, where a segment of one parent is copied into the child and the remaining cities are filled in their order of appearance in the other parent; no operator dominates on TSPLIB.

2.3 Motivation of the Developed Approach

The evolutionary approach we develop follows the genetic algorithm presented in Section 9.2 of Mitchell [6], adapted to the permutation representation of the TSP; its pseudo-code is given in Section 3. Within this template, the design choices left open concern the initialisation of the population and the crossover operator, which we align with the literature reviewed in Section 2.2. We test four initialisations: random, greedy nearest-neighbour tours refined by 2-opt [2], geometric non-crossing tours [3], and their combination. For recombination we adopt the order crossover (OX) [4]. The remaining operators are left unchanged, so that the effect of the two adapted components can be isolated in Section 4.

3 Algorithm

Each individual is represented as a permutation of all cities, where the order of the cities defines the tour and the final city is connected back to the first. The initial population consists of p tours generated by the selected initialisation method. The cost of an individual is given by Equation (4). Since the TSP is a minimisation problem, this cost is converted into a maximisation-based fitness function defined as

$$F(\pi) = \frac{1}{C(\pi)}. \tag{5}$$

Consequently, shorter tours receive higher fitness values and are favoured during selection. In the implementation, tour costs are used directly where possible, since minimising $C(\pi)$ is equivalent to maximising $F(\pi)$.

In each generation, approximately a fraction $(1-r)$ of the population is retained for the next generation: each survivor is chosen independently from the current population by tournament selection of size k, and is copied to the new population before mutation. The remaining fraction is filled with offspring generated from selected parent pairs.

Parent selection for crossover is performed using tournament selection. For each parent, k individuals are sampled randomly from the current population, and the individual with the lowest tour cost is selected. The process is repeated to obtain the second parent. Tournament selection provides selection pressure towards shorter tours while still allowing different individuals to participate in reproduction.

Order Crossover (OX) is used to generate offspring because it preserves valid permutations. A segment from one parent is copied to the offspring, while the remaining cities are inserted

according to their order in the second parent, skipping already used cities. Two offspring are generated for each parent pair by swapping the parent roles.

After crossover, mutation is applied to a fraction m of the new population. The individuals to be mutated are selected uniformly at random. The implementation supports swap and inversion mutation; inversion mutation is used in the experiments described here. Swap mutation exchanges two randomly selected positions in a tour, whereas inversion mutation reverses the order of the cities within a randomly selected segment. Both mutation operators preserve a valid permutation.

Once mutation has been applied, the new population replaces the previous one and all tour costs are evaluated again. The best tour found so far is stored throughout the run. This means that even if a previously good individual is later modified by mutation or disappears from the population, the best solution encountered during the search is not lost.

The process is repeated for G generations. After termination, the best tour encountered during the complete run is returned.

Algorithm 1 Genetic Algorithm for the Travelling Salesman Problem

Require: cities D, population size p, replacement rate r, mutation rate m, number of generations G, tournament size k
Ensure: best tour found
  $P \leftarrow$ generate p initial tours
  evaluate the cost of each individual in P
  best $\leftarrow$ individual in P with minimum cost
  for $g = 1$ to G **do**
    create new generation $P_s \leftarrow \emptyset$

    Selection:
    $q \leftarrow \text{round}(rp/2)$
    $s \leftarrow p - 2q$
    for $i = 1$ to s **do**
      select an individual from P using tournament selection of size k
      add it to P_s
    end for

    Crossover:
    for $i = 1$ to q **do**
      select h_1 from P using tournament selection of size k
      select h_2 from P using tournament selection of size k
      $o_1 \leftarrow \textsc{OrderCrossover}(h_1, h_2)$
      $o_2 \leftarrow \textsc{OrderCrossover}(h_2, h_1)$
      add o_1 and o_2 to P_s
    end for

    Mutation:
    uniformly select mp individuals from P_s
    for each selected individual π **do**
      apply the selected mutation operator to π
    end for

    Update:
    $P \leftarrow P_s$

    Evaluation:
    evaluate the cost of each individual in P
    if the best individual in P has lower cost than *best* **then**
      update *best*
    end if
  end for
  return *best*

4 Experimental part

4.1 Hardware and Software

All experiments were run on two MacBook Pro laptops with Apple Silicon (M-series) processors. The algorithm is implemented in Python 3.12 using only the standard library; `matplotlib` is used solely to produce the figures. Every trial runs as a single process. Random numbers are

generated from explicitly assigned seeds, making the experiments reproducible with the same code and configuration.

4.2 Experimental Protocol

Each configuration is repeated for 30 runs with the exception of kroA200 parameter sweeps, which use 10 runs. The study proceeds in three stages. First, random, nearest-neighbour tours improved by 2-opt, angle-based tours, and their combination are compared on burma14, berlin52, kroA100, and kroA200. Second, population size $p \in \{50, 100, 120, 150\}$, number of generations $G \in \{100, 300, 500, 800\}$, mutation rate $m \in \{0.05, 0.1, 0.2, 0.4\}$, and replacement rate $r \in \{0.3, 0.5, 0.7, 0.9\}$ are varied one at a time on the same four instances and for all four initialisation methods. The remaining parameters are held at $p = 100$, $G = 500$, $m = 0.1$, and $r = 0.7$, and inversion mutation is used. Third, the problem-size experiment uses random initialisation and these default parameter values on all six instances.

4.3 Datasets

The evaluation uses six symmetric TSPLIB instances [5] of increasing size: burma14 (14 cities), berlin52 (52), kroA100 (100), kroA200 (200), pcb442 (442) and pr1002 (1002). They are committed to the repository and loaded with their original TSPLIB edge weights (GEO for burma14, EUC_2D for the others). Performance is measured as the optimality gap $(C_{\text{best}} - C_{\text{opt}})/C_{\text{opt}} \cdot 100$, using the published optima 3323, 7542, 21282, 29368, 50778 and 259045. The main study uses the first four datasets, while the scaling experiment covers all six instances.

5 Results and Analysis

5.1 Effect of the Initialisation

First, we compared the effect of the initial population by testing four initialisation methods on the same four TSPLIB instances with the same parameter settings.

Instance	Random	NN + 2-opt	Angle-based	Smart
burma14	0.50 ± 1.34	0.00 ± 0.00	0.20 ± 0.20	0.00 ± 0.00
berlin52	9.20 ± 3.09	0.00 ± 0.00	10.71 ± 3.73	0.00 ± 0.00
kroA100	44.52 ± 7.20	1.55 ± 0.50	24.46 ± 4.76	1.88 ± 0.14
kroA200	162.69 ± 8.65	2.08 ± 0.28	70.99 ± 5.23	1.96 ± 0.34

Table 1: Mean final optimality gap (%) $\pm$ sample standard deviation over 30 runs for each initialisation method and instance. Lower gaps indicate better solutions.

The nearest-neighbour and smart methods already reached the known optimum in their initial populations on burma14 and berlin52. On kroA100 and kroA200, their mean initial gaps were about 1.91% and 2.25%, respectively, and the GA reduced these gaps by less than 0.4 percentage points. In comparison, the random and angle-based initialisations began further from the optimum, but the GA reduced their gaps by a substantial amount during the search. This shows that the initialisation method had a strong effect on the final solution quality.

5.2 Parameter Sensitivity

Table 2: Mean gap (%) with random initialisation when one parameter is varied and the others are kept at their default values (in bold). 30 runs per cell (10 for kroA200). bur14 = burma14, ber52 = berlin52, kA = kroA.

	Value	bur14	ber52	kA100	kA200
p	50	0.7	16.9	80.5	228.2
	100	0.6	9.8	46.6	155.2
	120	0.4	8.9	39.3	144.2
	150	0.3	9.0	32.2	126.5
G	100	0.4	38.8	175.6	431.0
	300	0.7	13.8	72.6	239.3
	500	0.6	11.3	44.7	154.8
	800	0.1	9.1	28.8	108.7

	Value	bur14	ber52	kA100	kA200
m	0.05	0.4	14.1	64.2	175.1
	0.1	0.5	10.0	44.3	159.1
	0.2	0.7	8.4	30.5	142.1
	0.4	0.0	9.0	24.4	151.1
r	0.3	1.0	11.9	57.7	202.8
	0.5	1.1	11.1	48.8	178.3
	0.7	0.7	9.9	45.8	164.6
	0.9	0.5	11.7	43.4	152.5

Table 2 reports the parameter sweeps on the four instances of the main study. Only random initialisation is shown, since with NN + 2-opt or smart seeding the mean gap on kroA200 stayed between 1.5% and 2.3% for every value tested. Each parameter was varied separately. The default configuration appears once in every block, each time with different seeds, and its results differ between blocks: 154.8–164.6% on kroA200 and 9.8–11.3% on berlin52. Results on kroA200 are based on 10 runs per setting, whereas the others are based on 30.

The number of generations and the population size had the clearest effect on the larger instances. Increasing G from 100 to 800 reduced the mean gap on kroA100 from 175.6% to 28.8%, and on kroA200 from 431.0% to 108.7%. Increasing p from 50 to 150 reduced it from 80.5% to 32.2% on kroA100 and from 228.2% to 126.5% on kroA200, while the running time grew roughly in proportion to p. Settings with a similar number of evaluations ($p \times G$) gave similar results on berlin52 and kroA100: 9.1% and 28.8% with $G = 800$, $p = 100$, against 9.0% and 32.2% with $p = 150$, $G = 500$. On kroA200 the gap was lower with more generations (108.7% against 126.5%). At $G = 800$ the gap was still decreasing on the two larger instances.

On the smaller instances these effects were weaker. On berlin52, populations larger than 100 gave gaps between 8.9% and 9.8%. On burma14 the gap stayed below 1.1% in every configuration, with slightly lower values for larger populations (0.7% at $p = 50$, 0.3% at $p = 150$).

A higher mutation rate gave shorter tours on the larger instances. On kroA100 the gap fell from 64.2% at $m = 0.05$ to 24.4% at $m = 0.4$. On kroA200 it fell from 175.1% to 142.1% between $m = 0.05$ and $m = 0.2$, and was 151.1% at $m = 0.4$; this difference is smaller than the spread of the default configuration noted above. On berlin52 the rate had little effect above $m = 0.1$. One possible explanation for the effect of mutation is that inversion reverses a segment of the tour, which is the same change as a 2-opt move, applied at random.

A higher replacement rate, meaning a larger share of new offspring in each generation, gave shorter tours on kroA100 (57.7% at $r = 0.3$ against 43.4% at $r = 0.9$) and on kroA200 (202.8% against 152.5%). On berlin52 the results showed no consistent trend, with gaps between 9.9% and 11.9%.

5.3 Scaling with Problem Size

To examine how the algorithm performs on instances of different sizes, we ran it on the six TSPLIB benchmarks using random initialisation with the same parameter settings. Table 3 reports the mean optimality gap, its sample standard deviation, the mean execution time, and the number of runs that reached the known optimum. A lower gap indicates a tour closer to the optimum.

Instance	Cities	Mean gap (%) ± SD	Mean time (s)	Runs at optimum
burma14	14	0.51 ± 1.16	0.75	18/30
berlin52	52	11.25 ± 3.27	0.81	0/30
kroA100	100	46.63 ± 6.23	1.48	0/30
kroA200	200	160.85 ± 12.38	2.69	0/30
pcb442	442	442.42 ± 12.28	6.29	0/30
pr1002	1002	1086.32 ± 16.16	14.72	0/30

Table 3: Optimality gap across 30 runs per TSPLIB instance using random initialisation. SD is the sample standard deviation of the recorded gaps and the mean time is measured in seconds; lower gaps are better.

The mean gap increased across the instances, from 0.51% to 1086.32%. The mean runtime also rose sharply with problem size, from 0.75s on burma14 to 14.72s on pr1002, showing that the search cost grows substantially as the number of cities increases. The algorithm reached the known optimum in 18 of 30 burma14 runs, but did not reach the optimum in any run on the other instances. This suggests that the GA struggled to find near-optimal tours on larger benchmarks.

6 Conclusions

We implemented and evaluated a genetic algorithm for the Travelling Salesman Problem on six TSPLIB instances. Our results showed that the initialisation method made a large difference, while the algorithm with random initialisation struggled on the larger instances. The parameter sweeps also suggested that using more generations and a larger population generally helped on the larger instances, while the effects of mutation and replacement rates were less consistent. In the future, we could try adding local search during the algorithm and test it on more instances.

References

[1] J.-Y. Potvin, "Genetic algorithms for the traveling salesman problem," *Annals of Operations Research*, vol. 63, no. 3, pp. 339–370, 1996. `https://doi.org/10.1007/BF02125403`.

[2] S. Liu, "A Powerful Genetic Algorithm for Traveling Salesman Problem," arXiv:1402.4699, 2014. `https://arxiv.org/abs/1402.4699`.

[3] Y.-F. Liao, D.-H. Yau, and C.-L. Chen, "Evolutionary algorithm to traveling salesman problems," *Computers & Mathematics with Applications*, vol. 64, no. 5, pp. 788–797, 2012. `https://doi.org/10.1016/j.camwa.2011.12.018`.

[4] A. Hussain, Y. S. Muhammad, M. N. Sajid, I. Hussain, A. M. Shoukry, and S. Gani, "Genetic algorithm for traveling salesman problem with modified cycle crossover operator," *Computational Intelligence and Neuroscience*, vol. 2017, Article ID 7430125, 7 pages, 2017. `https://doi.org/10.1155/2017/7430125`.

[5] G. Reinelt, "TSPLIB—A Traveling Salesman Problem Library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991. `https://doi.org/10.1287/ijoc.3.4.376`.

[6] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997, ch. 9, sec. 9.2. `https://www.cs.cmu.edu/~tom/files/MachineLearningTomMitchell.pdf`.